AI & AUTOMATION ENGINEER / AGENTS

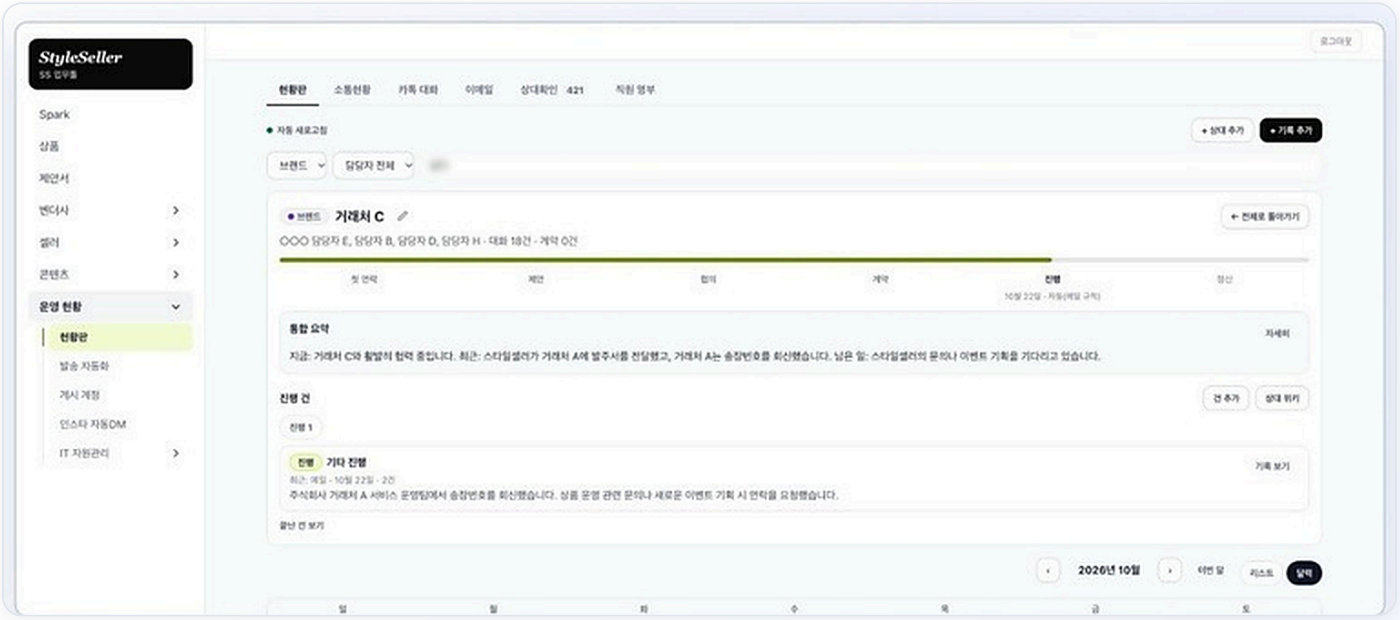
Portfolio

“Turning repetitive field work
into AI systems that run every day”

Jinhee Mok

AI & Automation Engineer · From design to deployment and operations

At StyleSeller I build, deploy, and run KakaoTalk and email record organization, vendor CS, internal work tools, and an agent platform. I don't call something done until I have run it on staff PCs.



Status board in production: stage bar, consolidated summary, and calendar for one counterpart (names blurred)

01

Status board Operations ledger

KakaoTalk, email, and Slack records
for hundreds of counterparts, one screen

02

Content Autopilot

From Reel request to approval
publishing defaults to manual after approval

03

Vendor CS Agent

Answers tagged questions only, no handoff
amounts and brands checked against evidence

04

Agent Platform · Spark

CEO routing, Jev (a judgment model)
for skill picks, tool gates, cost caps

05

Knowledge Hub Email Agent

13 LightRAG workspaces
only evidence-backed drafts reach the assigned staff member

06

Internal Work Tool

Pricing calculated by code
automated personalized KakaoTalk sends

07

Staff PC Deployment & Ops

One-line install across 11 PCs,
with per-device keys.

Jinhee Mok
Jinheemok815@gmail.com

STYLESELLER
https://jin-tonix.github.io/

Jinhee Mok

jinheemok815@gmail.com · 010.9030.2888
jin-tonix.github.io
Yeoksam-dong, Gangnam-gu, Seoul

AI & Automation Engineer · From design to deployment and operations

Resume

Summary

I replace work that staff used to repeat by hand with AI agents and automation, and I run it myself. Since November 2025 at StyleSeller, I have designed, built, and operated vendor CS, email-reply, and internal data-query agents, an internal work web tool, and deployment to staff PCs. After studying law, I worked for 8 years as a daycare operations manager and an English teacher, then moved into development. I hand much of the coding to an AI coding tool (Claude Code). I set the scope and check the results myself.

Experience

StyleSeller · AI & Automation Engineer (PL) — 2025.11 ~ present (about 1 year)

- LLM agent design, development, and operations: vendor CS agent, email-reply agent, internal data-query assistant
- RAG and knowledge search: LightRAG (Neo4j + pgvector) knowledge hub, claim-by-claim evidence checks on drafts
- Work automation: KakaoTalk and email collection, product registration from supplier catalogs, price calculation and partner matching
- LLM evaluation and improvement: skill router accuracy 90.15% (132 questions), re-measured after fixing answer-key errors and a fake skill
- Deployment and operations: GCP VM, Docker, NGINX; one-line install and auto-update deployment to 11 staff PCs; same-day recovery from a DB outage and move of 4 DBs to self-hosting

We:Review · AI Development Intern — 2025.08 ~ 2025.10 (3 months)

- Receipt processing automation: structured data extraction with PaddleOCR, DBSCAN, and Qwen2.5-7B, served with FastAPI
- Took part in building the receipt-review pipeline and its live service deployment: review accuracy 70% → 95%, review time cut by 85%

Jeongsang Language School · Pots&Pans · Montekids English Academy · English Teacher — 2018.08 ~ 2024.02

Designed English immersion classes for ages 5 to 15, wrote lesson plans, ran individual study plans for students, and held regular parent consultations.

Jeonwon Daycare Center & After-School Program · Operations Manager — 2015.03 ~ 2018.02 (3 years)

Teacher hiring, annual curriculum and event planning, communication with parents and the steering committee, and day-to-day operations as acting director.

Education

Cheongju University · Law — 2008.03~2012.02

Academic Credit Bank System, Business Administration — 2021.02~2023.06

Training

Himedia Academy · Full-Stack (960h) — 2024.04~10

Himedia Academy · Advanced AI — 2024.10~12

Korea IT Business Promotion Association · Generative AI — 2025.05~07

Skills

AI·LLM	Gemini, LiteLLM, LightRAG, RAG, tool calling, MCP, evaluation-set design and measurement
Backend	Python, FastAPI, TypeScript, Node.js, PostgreSQL
Infra	GCP, Docker, NGINX, Vercel, GitHub Actions, n8n

At a Glance

Seven systems I built and run at StyleSeller, plus one follow-up improvement. Each case explains how I built it and why I decided it that way. Figures are stated as of each case's own reference date.

Status board · Operations ledger

Collects KakaoTalk, email, and Slack records into a per-counterpart calendar and shows deal stages for hundreds of counterparts on one screen

Content Autopilot

From Reel request through render and approval, all in the staff screen; publishing defaults to manual after approval

Vendor CS Agent

Answers only tagged questions in vendor group chats, no handoff, and checks amounts and brands against evidence in code

Agent Platform · Spark

Starts from a CEO work order, with the server controlling the dispatcher, workers, shared skills (Jev recommendation), and tool gates

Jev Skill Router Upgrade (Spark follow-up)

Made the Hermes agent's skill selection (Jev) accurate and fixed measurement errors. Estimated that letting Jev filter first could cut cost to about 1/10 (not measured)

LightRAG Knowledge Hub · Email Agent

Four agents find evidence from the same knowledge hub, and only evidence-backed drafts reach the assigned staff member's Gmail drafts folder.

Internal Work Tool

Moved spreadsheet pricing formulas into code; the LLM only returns the input values to change. Automated personalized KakaoTalk sends

Staff PC Deployment & Operations

Deployed KakaoTalk and Instagram tasks to 11 staff PCs with a one-line install

Status board · Operations ledger

Pick one counterpart and KakaoTalk, email, and order sheets land on one calendar, with deal stages for hundreds of counterparts on one screen

Period	2026.10.19–10.23 (5 days), in production.
Role	Design: ledger schema (a table that records each fact only once) and counterpart-linking rules / Build: intake API, mail collector, 6 screen tabs / Verification: DB tests, production screens. Used an AI coding tool, set the scope myself, and confirmed results through testing
Repo	About 150 status-board-related commits across 2 internal repos

Background

Conversations with counterparts are split across KakaoTalk rooms, the order mailbox, and Slack, and Excel order sheets arrive separately as email attachments. The status board summarizes these records by day and puts them in date order on each counterpart's calendar. Pick a brand, vendor, or seller, and every record exchanged with that counterpart shows up on one screen. The Communication status tab shows which stage each of hundreds of counterparts is in, from First contact to Settlement. Records the rules can't link, or stages that are ambiguous, are left for staff to decide.

Structure and Scale

1. Tech

React 19.2 · Tailwind 4 · Vitest 4, Python 3.12 · Pydantic v2, Postgres RLS (a DB rule that limits who can see each row), Gemini Flash family (via LiteLLM)

2. Scale (dated per item)

- Communication status stage mix: First contact 13.1%, Proposal 33.8%, Negotiation 2.6%, Contract 4.5%, In progress 46.0%, Settlement 0% (2026-10-24)
- About 100–200 records per weekday, hundreds waiting in Counterpart check (2026-10-24)

3. How it works

① KakaoTalk comes in 3 times a day (09:15, 13:00, 17:55), the order mailbox every 5 minutes, and Slack through a general-purpose connector. However many times the same record arrives, the ledger keeps it only once.

② Newsletters are filtered out by rule before the LLM ever sees them, and in order-sheet tables, the recipient and orderer columns are found and those names are blurred. If a summary's cited phrase isn't an exact substring of the masked text, the summary is dropped.

③ No LLM is used to link counterparts — only rules. It checks, in order, mail thread, business registration number, KakaoTalk room, email, phone, domain, vendor code, and alias; if the keys disagree, the record stays unclassified.

④ In-progress deals group KakaoTalk contract signals and email signals by counterpart · target · round. A stage is applied automatically when the rule evidence is clear; the rest go to "awaiting review" for staff to decide.

⑤ Every day at 18:05, summaries for 500+ counterparts are rewritten into 3–6 lines. If the daily cost passes KRW 2,000, or the cost ledger can't be read, that day's batch stops.

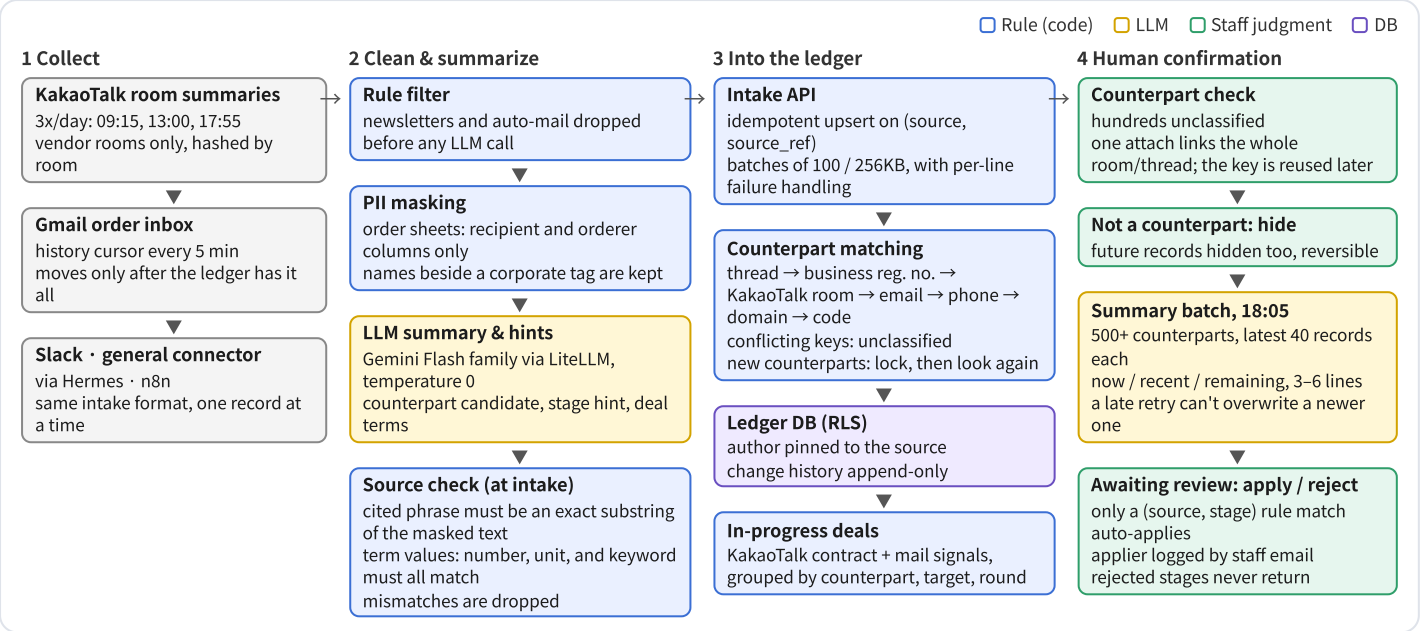
Incidents and Fixes

1. **All 107 summaries rejected (2026-10-23).** The DB function that saves summaries lacked execute permission for the app role, so all 107 summaries that day failed at the save step with a db_error. Fixed with a single GRANT line scoped to just that function.

2. **Over-eager "in progress" suggestions in a test run (2026-10-23).** The rule treated words like "shipping," "inventory," or "sales" alone as a deal signal. Changed it to keep only genuine deal-signal words and first strip non-deal phrases like "discontinued" or "order inquiry" before matching.

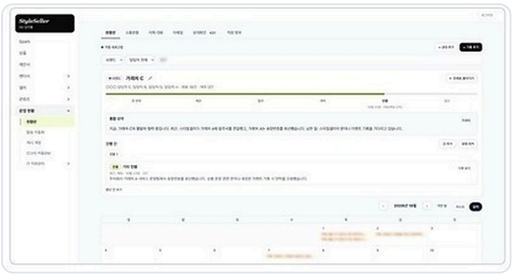
3. **A shared chatbot shown as the owner (2026-10-23).** The owner field for 64 counterparts showed a shared chatbot's name. The top 5 most active speakers in a KakaoTalk room were taken as the owner, and the chatbot was among them. Now only a company email account (@sample-corp.co.kr) is accepted as owner, falling back to the counterpart profile's owner if there isn't one.

Architecture

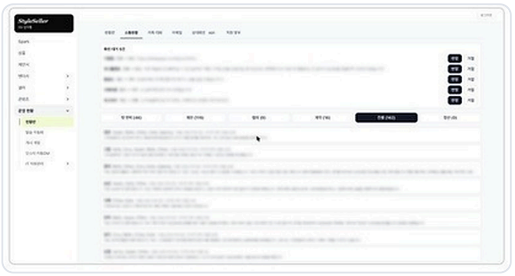


KakaoTalk, email, Slack → server (masking, LLM extraction, source check) → idempotent intake → ledger (RLS, rule matching) → staff confirmation

Screens (Korean UI)



Screen A: per-counterpart calendar. KakaoTalk summaries, emails, and Excel order sheets land on one dated calendar, under the stage bar, consolidated summary, in-progress deals, and counterpart wiki.



Screen B: Communication status. Counterparts by stage (13% First contact to 46% In progress) and "awaiting review" suggestions.

Technical Highlights

1. A resent record changes nothing. One upsert on (source, source_ref) stores it and leaves staff-edited or unchanged rows alone; identical content doesn't bump the sequence or timestamp.

2. The caller never sets the author. The intake function discards it and sets "ingest:<source>". The "manual" source is staff-only. A trigger stamps "edited by staff", and a hidden record can't be revived with UPDATE. The app role can only execute functions.

3. Mismatched keys are never linked. The order is fixed: thread, business reg. no., KakaoTalk room, email, phone, domain, vendor code, alias. If a business reg. no. or phone points elsewhere, it is flagged as a conflict and left unclassified. 13 shared domains such as gmail and naver are never evidence.

4. Concurrent collection never creates a counterpart twice. Creating one takes an advisory lock (a transaction-scoped name lock) per evidence value, in sorted order, then looks again, using any counterpart another transaction just created. Staff stage changes use the same lock order, so no deadlock.

5. LLM output is checked against the source. A cited phrase is kept only if it is an exact substring of the masked text. Term-condition values are stricter: each number must appear in the quote, and the keyword and unit must match. A company name must appear, normalized, in the evidence line.

6. Order sheets are masked by column. Header columns such as "recipient" or "orderer" are masked only on rows whose cell count matches the header. Product names and quantities stay. A name next to "(Co., Ltd.)" counts as a company and is not masked.

7. Cost is reserved before the call. The balance check and the reservation are one SQL statement, so two concurrent calls can't both pass the cap. The reservation is the larger of an input/output estimate and a minimum. If cost posting fails, later calls stop.

8. The mail cursor moves last. Mail goes to a local queue, then the ledger. The Gmail cursor advances only after the whole batch succeeds; if it expires, the list is re-read from the last successful run. Mail that fails twice is quarantined with an alert.

Status, Judgment, Limits

- In production: 6 tabs, KakaoTalk/email/Slack intake, and the 18:05 batch (production screen checked 2026-10-24).
- Python, TS, and SQL share one golden-test fixture for name/phone normalization. CI runs content tests on PGlite under the real app role. Screen tests run outside CI.
- Limit: I haven't measured how often staff apply "awaiting review" suggestions, so suggestion accuracy is unknown. The server doesn't re-check cited phrases; it trusts the intake-side check.
- Next time: a test that calls every new DB function as the app role. It would have caught the 10-23 rejection of 107 summaries before release.

What I Decided

- Counterparts are linked by rule order, not by an LLM. If keys disagree, nothing is auto-linked; staff decide.
- Automatic stage application is switched on one (source, stage) rule at a time. The first: KakaoTalk "shipped / ordered / group buy" → "In progress". LLM self-confidence isn't calibrated, so I don't use it.
- A stage staff rejected is never suggested again, so the queue holds only suggestions nobody has judged.

Content Autopilot

StyleSeller · internal, private repository

Covers a Reel request through a rendered 9:16 MP4 to approval in the staff screen. On this new path, live SNS posting has not started yet.

Period	2026.10.03–present (after the design doc was approved)
Role	Design: stage state machine, publishing defaults. Build: orchestrator, publish gate, DB slots, QA gate. Verification: production canary (a first test run of one item) and restart-scenario tests. I used an AI coding tool, set the scope, and checked the results myself
Repo	Orchestrator, content agent, video renderer, work tool (content screen)

Background

A staffer requests a Reel in the work tool, reviews a preview, approves it, and schedules the posting time. Generation costs money and publishing is public, so every default is the safe one: publishing is manual, recurring automation is a dry-run (it goes through the motions without executing), and paid generation is off. Code, not a person, moves each stage forward.

Structure and Scale

1. Tech

Paperclip sub-issues, orchestrator (state machine, no LLM), Hermes and Gemini Flash family (via LiteLLM), image-service, TTS, video renderer, HyperFrames (renders video from HTML), ffprobe, Google Drive, Blotato (SNS scheduling service), Supabase (PostgreSQL)

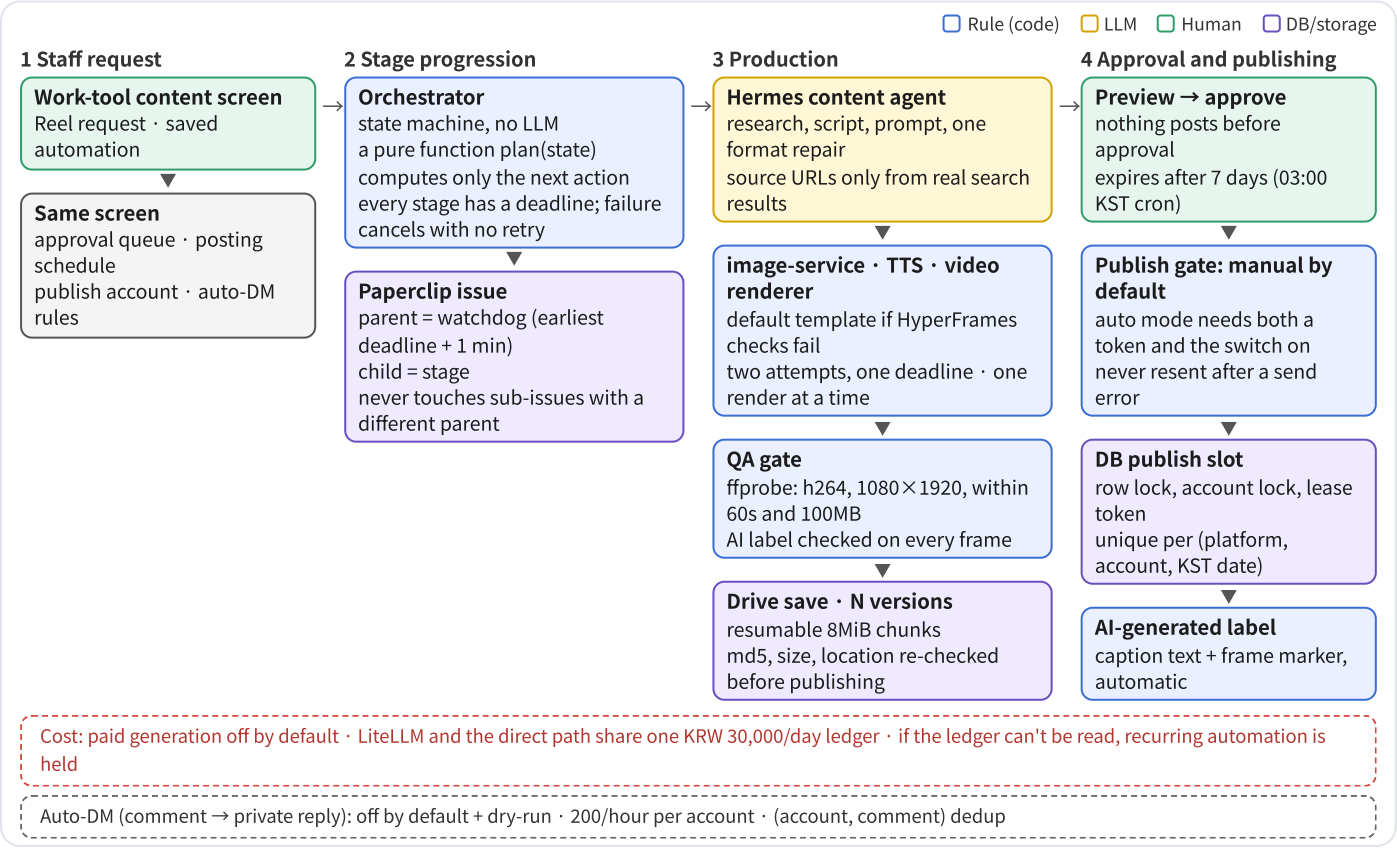
2. Scale (dated per item)

- 3 services, 122K lines, 617 test files (orchestrator 135, Hermes agent 396, video renderer 86)
- Production canary verified through saving a 1080×1920 MP4 to Drive (2026-10)
- 0 live SNS posts from this new path. The DB allows one post per account per day; the publish gate caps at 10 per day (2026-10-24)

3. How it works

① A staffer requests a Reel or runs a saved automation (immediate, once, or recurring).
② A pure function in the orchestrator computes only the next action. The executor applies it to a Paperclip sub-issue.
③ The Hermes agent writes research, script, and prompts. The video renderer makes a 9:16 MP4 from a HyperFrames template.
④ A version that passes the QA gate is saved to Drive with a checksum. A person approves it from a preview. A pending approval expires after 7 days.
⑤ Publishing is manual by default: download the MP4, copy the caption, click "published." The "AI-generated content" label and frame marker are added automatically.

Architecture



Request → state machine → Hermes script → render/QA → human approval → publish gate (manual by default, DB slot)

What's Blocked by Default

1. Publishing. Defaults to manual. Auto-publish needs both a token and the switch on. A partial unique index on (platform, account, KST date) allows only one post per account per day and rejects any more. A migration halts first if existing duplicates are found.

2. Paid generation and recurring automation. The paid backend defaults to False. Recurring automation defaults to dry-run and only unlocks when the dry-run setting is exactly "false". It reads the company's KRW 30,000/day ledger before starting, and holds if it can't read it. The cap is clamped with min(), so an env var cannot raise it.

3. Instagram auto-DM. The on-switch defaults to off, and dry-run defaults to on. Limited to 200/hour per account, and a malformed setting is read on the safe side. A (account, comment) unique index stops a DM going to the same comment twice.

Technical Highlights

1. A pure function decides the next action. plan(state) only returns a list of actions; the executor applies them. Every stage has a deadline, and the parent issue wakes at the earliest deadline + 1 minute to watch. On failure it cancels and closes the sub-issue with no retry. It never touches sub-issues with a different parent, so re-running never spins up a new pipeline.

2. A format error only gets one repair attempt. The repair target is JSON format errors only. It sends the error list (max 8 items, 120 chars each) without the prior output, and calls at most twice. A test confirms that a failure after the repair ends the run, with no third call.

3. Sources are only accepted from real search results. A cited URL only passes if it's in the web tool's results or the input; an evidence field the model wrote itself is discarded. Merely claiming a search was done fails as web_research_unverified. Facts like product price are still blocked only by prompt rules for now.

4. A post is never resent when the outcome is unknown. Right before the POST, a start marker is written; if an error follows, it's set to publish_pending instead of failed. If it turns out to have already been published on restart, it closes as already_published. Blotato's submission ID is saved immediately, so waking up again only queries status.

5. The DB hands out publish slots. A claim function locks the row with FOR UPDATE, takes an account advisory lock, checks the daily cap, then issues a 30–900 second lease token (a temporary lock marker). Chose this over an in-process memory lock so the decision is made in one place in the DB no matter how many processes are running.

6. One ledger enforces two policies. To close the hole where the direct path bypassed the proxy's cap, LiteLLM and the direct Vertex path add cost to the same Redis key. If the ledger can't be read, an individual call still goes through, but starting recurring automation is held. Reads use a Redis user with no write permission.

7. Render retries share one deadline. If the motion plan the LLM wrote fails the HyperFrames check, it re-renders with the default template. The two attempts share one deadline, so a retry doesn't double the wait time. Rendering runs one at a time.

8. The file to publish is pinned by checksum. Right before publishing, it re-checks the Drive file's mime type, size, md5, and location. The QA gate uses ffprobe to confirm h264 at 1080×1920 within 60 seconds and 100MB, and checks the AI label on every frame of an AI video. Uploads read in 8MiB chunks and resume on a 308 response (the resume signal).

Status, Judgment, Limits

- This new production path is technically complete, but details like the prompts inside it are still being tuned. Content made through this path has no live SNS track record yet; the production canary has been verified through saving an MP4 to Drive (2026-10). Existing videos are already being published live, through a separate path that only swaps the caption and hashtags per account and posts to several accounts at once.

- Approval expiry runs on a daily 03:00 KST cron, in order: reject, cancel the issue, close the row. Even if canceling the issue fails, the row still closes, so the same failure doesn't repeat.

- Limits: product-fact verification is still prompt rules only, and registering a new Instagram token needs a developer to configure it. This path has no live posts yet, so I haven't measured outcomes like views.

- If I did it again: I'd add a code check that cross-references product facts like price and specs against the product DB. Right now, if the model breaks a rule, a human has to catch it in preview.

What I Decided

- Stage progression is owned by a state machine with no LLM; the LLM is only used for research, script, and prompts. Handing the next step to the model would make results non-reproducible across re-runs.

- Nothing auto-retries beyond one format repair and one default-template re-render; everything else closes. Anything that costs money or goes public needs a human to press it again.

- Publishing defaults to manual, and the daily cap lives in a DB index, not app config.

Vendor CS Agent

StyleSeller · internal, private repository

A KakaoTalk chatbot that answers only tagged questions, with no handoff, and corrects numbers and brand names against evidence

Period	2026.06–present
Role	Design: collection path, vendor gate, evidence-check rules. Build: server agent, send and delivery confirmation in the kakaocli fork (a CLI that reads the Mac KakaoTalk DB and sends via Accessibility), Windows collector. Operations: incident response, the 09-02 response-style switch. I used an AI coding tool, set the scope, and verified by testing and measurement
Repo	Server (vendor agent), kakaocli fork, CS Copilot extension · local sender

Background

Staff PCs collect conversations and upload them to the server. Replies are sent from staff Macs. The server agent searches each vendor's records and the LightRAG knowledge hub (an entity/relation graph RAG, 13 workspaces) to write answers. Since 2026-09-02 it no longer asks clarifying questions: it answers only tagged questions, with no handoff. Instead it checks amounts, percentages, and brand names against evidence, and searches and rewrites on a mismatch.

Structure and Scale

1. Tech

macOS kakaocli fork (Swift), Windows read-only collector (Python), Python 3.12, Gemini Flash family (via LiteLLM), LightRAG 1.5.6, Google Sheets · Drive

2. Scale (dated per item)

- Vendor-agent code, tests, and config: 61K lines combined, 600 commits, 107 test files (2026-09-23)
- CS Copilot extension · local sender: 631 commits, 20,596 lines, 56 tests (2026-09-23)
- Tests: vendor gate 20, evidence judging 16, liveness markers 30, Mac Swift 34, Windows Python 72

3. How it works

① On Mac, kakaocli reads only what follows the last logId in the KakaoTalk DB. On Windows, a read-only collector uploads. Each moves on only after the server answers 2xx.

② The server drops a repeat of the same message within 15 seconds. It keeps per-vendor ledgers, sheets, and Drive files, builds a wiki twice a day (09:30, 18:10), and feeds the knowledge hub.

③ It checks the vendor ID or name in the room name against the work tool's list of served vendors. If the vendor isn't on the list, it doesn't answer.

④ It accepts only questions tagged @company_AI. The regex only detects the tag. The model searches and writes. Other vendors' names are masked as [other vendor].

⑤ It checks the draft's amounts, percentages, and brand names against evidence, and rewrites on a mismatch. A staff Mac sends the reply. Delivery is confirmed when the KakaoTalk DB count rises.

Incidents and Fixes

1. A rule hijacked the decision (through 2026-08-30). The trigger regex also did the blocking, so some messages were silently blocked with no LLM judgment. I cut the regex back to tag detection and moved the answer-or-not decision to the model.

2. Sends got stuck (2026-08-17–18). On macOS 15, with the KakaoTalk window in the background, the Accessibility (AX) call ignored its timeout and hung. 31 of 147 sends (21%) failed permanently. I moved the call to a separate thread and cut the wait off at 3 seconds with a semaphore. Since 09-08, delivery is re-confirmed after every send.

3. A flood of past conversations (2026-09-16). An old collector build uploaded whole conversation histories. 3 Windows PCs sent 2,500 messages in 50 minutes, the Sheets API returned 429s, and the 2 PCs installed afterward saw empty rows for over an hour. The server now discards anything before a reference date and answers 200. A 5xx would make the collector re-upload the same records.

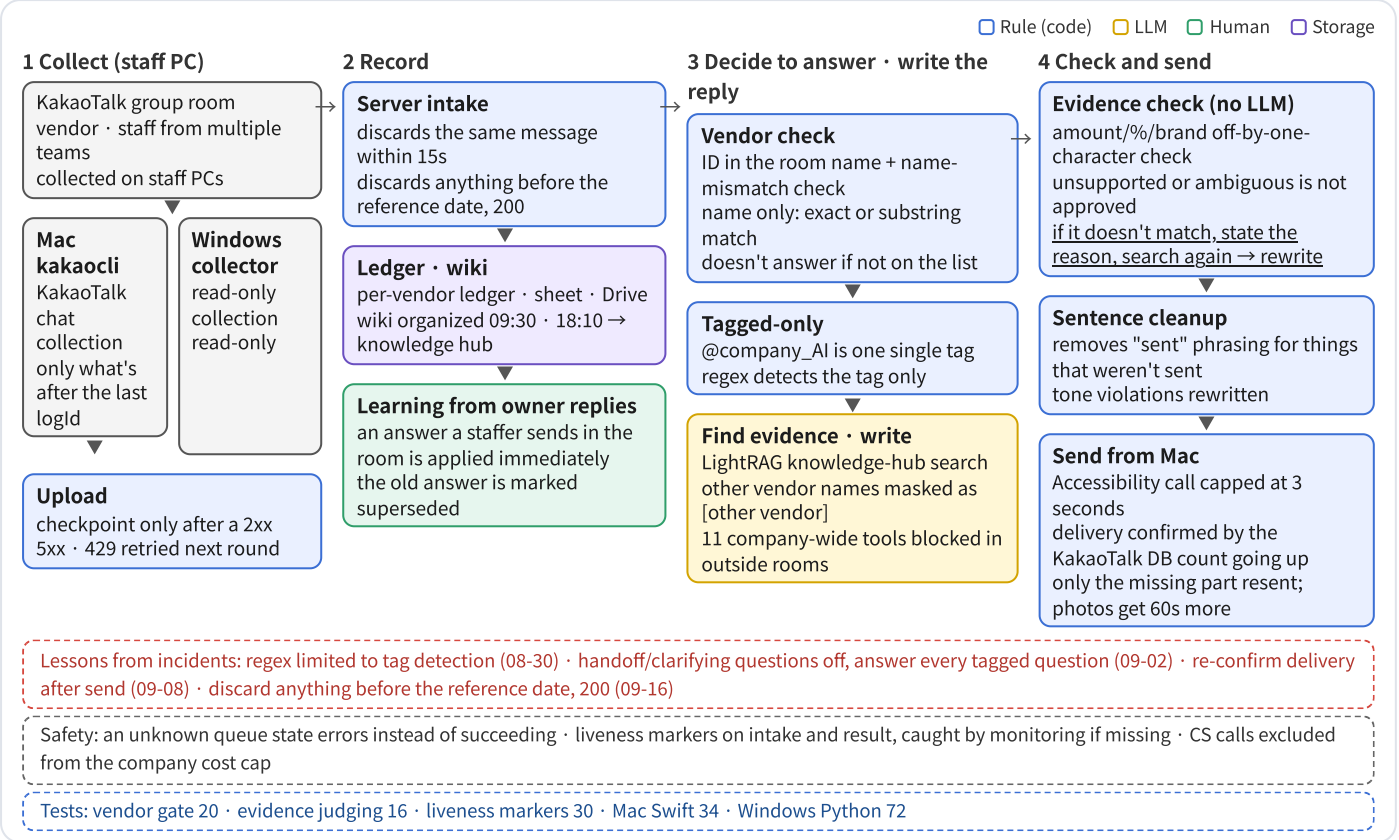
Status, Judgment, Limits

- Collection, ledger/wiki building, and tagged-question answering have run since 2026.06. An answer a staffer sends directly in the room is learned at once, and the old answer to the same question is marked superseded.
- Vendor and influencer CS calls are excluded from the company's daily cost cap. The Windows collector has no auto-update, so a new build is deployed by reinstalling.
- Next time: add auto-update and a reference date to the Windows collector from its first build. The 09-16 flood came from the old build, and reinstalling was the only fix.

What I Decided

- On 2026-09-02 I turned off clarifying questions and switched to answering only tagged questions, with no handoff. In exchange, number and brand checks run in code, not in the LLM. The old handoff branch stays in the code, guarded by a test.
- The regex only detects tags and the model decides. The setup where the regex also blocked was removed on 2026-08-30.
- I dropped fuzzy matching for vendor identification; Not answering is better than answering in someone else's room because a name looked similar.

Architecture



Staff PC collection → server ledger · wiki → knowledge hub → writes an answer to a tagged question → checks evidence · rewrites → sends from Mac · confirms delivery

Technical Highlights

1. A similar name never decides the vendor. The "(1234)" prefix in a room name is read as the ID. If the name part matches a different vendor with 0.80+ jamo similarity, it's treated as an ID typo and blocked. With no ID, only an exact or substring-matching name is accepted. Common suffixes alone scored 0.81–0.88, so fuzzy matching was dropped.

2. One message arrived 51 times. kakaocli resends the latest message in an unread room every 2 seconds. Counted it out: 51 resends of that one message, while the collector sent 170–200 messages per minute. Drops resends within 15 seconds keyed on (room, message, unread count, sent-by-me). The unread count is part of the key, so closing and reopening the room still gets through.

3. Numbers and brands are checked in code. Amounts and percentages in the draft are normalized ("30k won" becomes 30000 KRW). A value not in the search evidence, or a word that differs by one character from the evidence brand, is flagged as a violation. When one turns up, it states the reason, searches again, and rewrites.

4. "I sent it" is only written when it's true. If the send API wasn't called that turn, a completed-tense sentence like "I've sent you the quote" is removed. On a retry with the same idempotency key, it checks real queue state. In-flight or already-sent items aren't re-queued; only failed ones are retried. If the state is unknown, it errors instead of claiming success.

5. Other vendors' names are masked. If a vendor name other than the current one shows up in a search result, it's swapped for [other vendor] and the model is told it was swapped. All 11 company-wide Spark tools are blocked in outside rooms. The vendor name used in a product recommendation lookup is set by the server, not the model.

6. Delivery is counted by the KakaoTalk DB count. A baseline count is taken before sending. After sending, it counts how many texts and photos appeared in the local DB and resends only what's missing. Photos get 60 more seconds of wait. In a 70-person room, a duplicate photo was judged worse than a missing photo. Reporting completion to the server also happens only after this check.

7. Checkpoints only move after a 2xx. The uploader only advances its offset after receiving a 2xx. 422 and 413 are skipped; a 5xx or 429 stops with no commit and retries next round.

8. The processing path leaves its own liveness markers. It stamps a marker once on accepting a tagged message and once on reaching a result. If no result marker shows up past a business-hours threshold, monitoring catches it. The old approach, where monitoring assumed "no draft means it's stuck" from outside, was dropped in review. This path has 30 tests.

Agent Platform · Spark

StyleSeller · internal, private repository

No matter who asks, the work starts from the CEO agent's work order, and the server controls delegation and tool execution

Period	2026.05 to present (kicked off 05-17, Spark/Work OS design 07-18~19)
Role	Design: work order, delegation guard, tool gate structure. Build: shared tool/skill box connection, one-time approval, LiteLLM cap, Jev router connection. Verify/operate: wrote the evaluation set and deployed to the VM. Used an AI coding tool, set the scope myself, and confirmed results myself
Repo	Internal repo (Paperclip fork), skill folder, service code

Background

An internal platform where agents hand issues to each other to get work done. Forked from Paperclip (an open-source agent issue manager); company-specific features are attached as HTTP services under a service folder outside the core. The only exceptions are Work OS (the work-status/approval/execution-log app) and Spark. Staff use Spark to ask read-only questions against 4 business DBs and hand the actual work to the CEO agent. The longer a delegation chain runs, the more a request can drift, and any single tool can touch money or external exposure — so control lives in server code, not in prompts.

Structure and Scale

1. Tech

Paperclip fork (Express 5, Drizzle, PostgreSQL), MCP SDK 1.29, Hermes (agent execution runtime), Python 3.12 with FastAPI, LiteLLM, Redis, Langfuse 4.50, Gemini Flash family (via LiteLLM)

2. Scale (as of 2026-10-24)

- About 5,000 of my own commits (excluding merges/upstream), about 2,200 test files (2026-10-24)
- 109 shared skills (as of 10-24), 66 trigger evaluation files (1,282 sentences), 67 tool manifests

3. How it works

① Spark, Slack, and brand-tool requests all go to the CEO agent. Only structured brand-tool requests go straight to the content orchestrator, and a work-order record is still kept.

② The CEO writes a v1 work order. It includes the original text attachment with a sha256, the goal, completion criteria, constraints, and an approval flag.

③ The Hermes dispatcher under the CEO hands the issue to one of up to 10 department heads; if the switch is off, it falls back to direct CEO delegation. The server caps delegation depth at 2 and delegations per work order at 5, and blocks an A→B→A bounce-back.

④ BM25 narrows candidates from the 115 shared skills, Jev re-filters them by gate and fit score, and Gemini picks one skill from the result. If Jev fails or times out, the Jev stage is skipped and Gemini picks directly from the full list. The upgrade process is detailed on page 13.

⑤ Tools run through a 6-stage gate and human approval. Publishing is owned by the Publisher gate (a single service dedicated to publishing); the default is manual, and a DB index blocks more than one post per account per day.

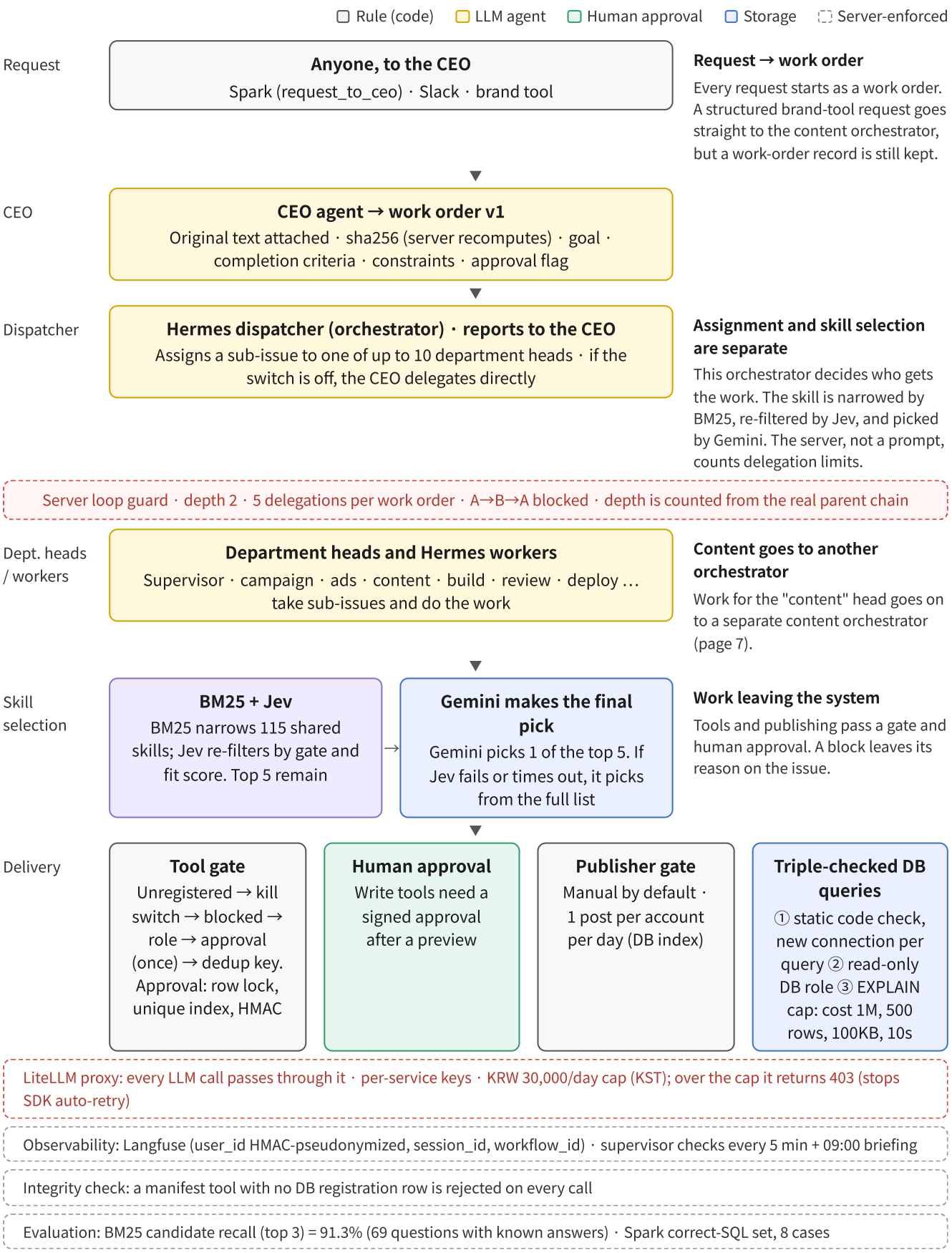
Incidents and Fixes

1. Spark went down entirely (2026-10-24). A new Spark tool was missing its DB registration row, which failed the boot check and caused every Spark call to be rejected — the system behaved exactly as designed when a check fails. After recovering with a restart, I added a migration rule that inserts a registration row for every tool, plus a regression test.

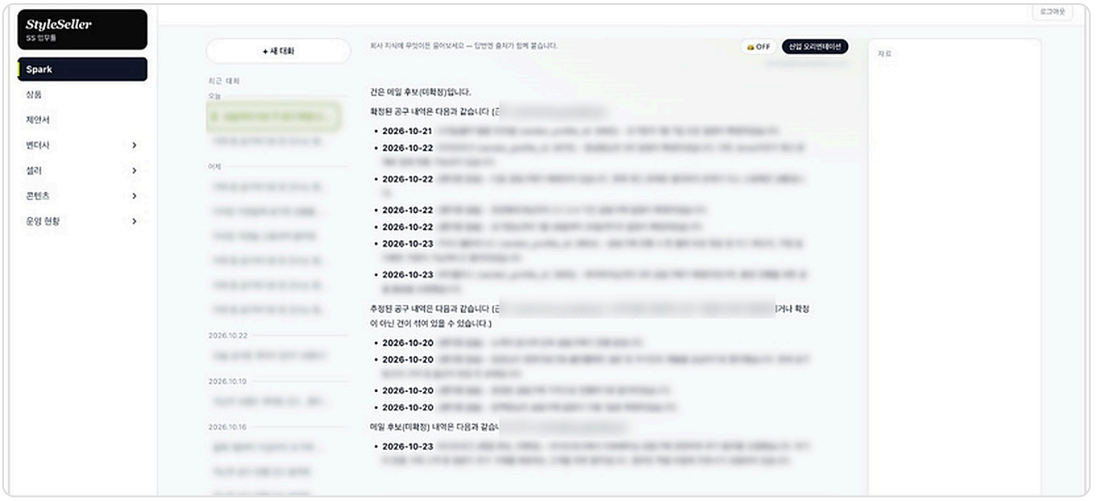
2. A still-rendering job got reclaimed (2026-10-24). The server's reclaimer saw a video job that had run over 5 minutes, treated it as stalled, and reclaimed it; the result that came back 7 minutes 40 seconds later was rejected with a 409. Now a liveness signal goes out every 120 seconds but stops once the issue's deadline has passed, so a job that's truly stuck still gets reclaimed. In production I confirmed the signal was being received (200) and reclaims stayed at 0.

3. Made the Jev router lightweight (2026-10-23). For speed, I also moved the Jev router onto a lightweight model, behind an adapter that only matches the response format — the existing router code was left unchanged.

Architecture



Screen



Spark's answer screen. Question titles and counterpart names are blurred; each answer carries the name of the tool that provided the evidence

Technical Highlights

1. The work order's original text is locked with a hash. The longer a chain of delegation runs, the more the request text can drift, so I NFC-normalize the original text, align line breaks, and lock it with a sha256. A hash sent by the client is overwritten by the server. I followed an external benchmark showing fewer supervisor errors when the original text is passed through instead of an intermediate summary.

2. The server cuts off delegation loops. A plugin only hooks in after an issue already exists, so it cannot see or stop a delegation that is still being created. I fixed this in the server core instead. Depth is capped at 2, delegations at 5 per work order, and A→B→A is blocked. Depth is counted by walking up the real parent issue chain rather than a column a caller could tamper with. A 6th delegation and a forged depth are both rejected by tests.

3. BM25 narrows, Jev re-filters, and Gemini picks. BM25 narrows candidates from the 115 skills, Jev re-filters them by gate and fit score, and Gemini picks one. On failure, timeout, or a tripped circuit breaker (300 seconds after 2 failures), the Jev stage is skipped and Gemini picks directly from the full list. Rollout targets are split by the first 8 characters of a sha256.

4. Approval is spent exactly once. The approval row is locked with FOR UPDATE, and a partial unique index blocks it a second time. Approval is consumed even if execution fails. The dedup key uses ON CONFLICT DO NOTHING followed by a re-read, so there's no race. Tests confirmed 1 execution out of 10 concurrent requests.

5. A write approval signs the preview. The input hash, preview hash, requester, execution ID, and expiry are bundled into canonical JSON and signed with HMAC-SHA256, so even DB write access isn't enough to get through without the key. The dedup key has no timestamp in it, so retries don't rack up extra approvals.

6. Spark's DB queries open a fresh connection every time. A read-only role alone wasn't enough. One set_config call can undo a timeout and read-only setting, and a reused connection would carry that state to the next caller. After filtering 23 dangerous functions, semicolons, and write CTEs, it opens one connection and closes it when done.

Status, Judgment, Limits

- In production. The company-wide switch for tool execution defaults to off, and each tool is turned on individually. user_id is sent to Langfuse as an HMAC pseudonym.
- Evaluation (as of the model used at measurement time): on 69 questions with known answers, BM25 candidate recall (the share of known answers that appear among the candidates) is 85.5% at rank 1 and 91.3% in the top 3 (2026-10-24).
- If I did it again: I'd check the manifest against tool registration rows in CI. The 10-24 outage only surfaced at boot, but it could have been caught before the merge.

What I Decided

- I chose a single full index over role-based skill bundles. On a 57-item selection set, both approaches scored the same accuracy, 0.947 (p=1.0). But the bundled approach missed skills that are only invoked by name. The cost difference over 66 items was KRW 27.
- The dispatcher handles assignment; skill selection is narrowed by BM25, re-filtered by Jev, and picked by Gemini. Even if Jev fails, Gemini still picks directly from the full list, so work doesn't stall. The pick is always made by Gemini, never by the worker.
- Controls like the loop guard, one-time approval, and the paid-call cap are all placed in the server and DB, so they hold even if the model tries to break them.

Follow-up · Jev skill router upgrade (next page)

- Separately from BM25 narrowing, I tuned a router that uses the same Jev rules and measured a final accuracy of 90.15% on 132 new questions — a different metric from the recall figure above.
- This approach was rolled into the Hermes agent, which is Spark's execution runtime.
- The real point wasn't accuracy itself — it was filtering out answer-key errors, fake skills, and inflated scores from the measurement.

Jev Skill Router Upgrade

Follow-up to the Spark case · 2026.10.25 ~ 10.30
StyleSeller · internal, private repository

To raise accuracy, I first had to fix how I was measuring it

Follow-up to the Spark case. Jev is a judgment model (TypeSafe) that returns probabilities, not text. The "Jev router" covers candidate narrowing through judging. Sub-agents that Hermes (Spark's execution runtime) assigns work to use it to pick skills. I rolled in the LLM-proposer approach I tuned this time. The earlier case's BM25 narrowing is as of 10-24. The skill list grew from 109 on 10-24 to 115 at test time. I designed the test, ran the experiments, checked results, and decided the rollout. I used an AI coding tool, but set the scope and verified results myself.

~90%
Accuracy 90.15% (119 of 132; 1 case = 0.76pp)

The number changed four times
Answer-key errors, inflated scores, a fake skill, re-measurement

Rolled out
Fully applied; 3/3 correct on real production questions

Request → BM25+Jev: narrow & verify (gate → fits) → Gemini: final pick → 1 skill, or "none"

The judgment model alone, with no BM25, topped out at 45-53%. Adding an LLM proposer on 10-26 reached the 80s. From 10-27 to 10-30 I worked toward 90%. The earlier case's 91.3% is BM25 candidate recall on a different test set, so it isn't comparable.

Corrections and re-measurement

The answer key was wrong 10-27
Of 6 cases flagged as "the router called a tool for a question that didn't need one," 5 were the router being right and the key being wrong. I would have misjudged this without opening each case myself. I kept the numbers from before and after the fix.

The setting I picked using the test set looked inflated 10-27 · 10-29
On contaminated data, the test set I had already seen gained +4.4pp and new questions +1.2pp. After cleaning and re-measuring, the seen-set gain became 0, and the new-set gain became +1.5pp. The gain was small and unstable either way, so I stopped using scores from the already-seen test set as evidence.

A fake skill was mixed into the test folder from 10-25 · found evening of 10-27
The same wrong answer appeared on two test sets, and I called it a "real bug" twice. I checked the 115 production skills one by one against the test folder and found a 116th that existed only there. Production could never make that error, so I withdrew the conclusion.

After the fix and rerun, the numbers shifted again 10-29
A cross-review noted that the comparison baseline was also measured on contaminated data, so I reran everything. I removed the fake skill from all 3 places and reran all 222 cases (90 + 132). The comparison setting, which I did not enable, picked a skill with a fit score as low as 0.22 whenever confidence was high, and got it wrong. The production setting never did, so I left the comparison setting off.

What was measured	Production setting	Comparison setting (not enabled)
① First measurement · 60 cases with known answers (answer-key errors still present, 10-27)	83.3%	91.7%
② After fixing the answer key · all 90 questions (10-27)	88.9%	93.3%
③ Re-measured without the fake skill · same 90 questions (10-29)	94.4%	94.4%
④ Same re-measurement · 132 new questions, reused afterward (10-29)	**90.15%**	91.7%

Production setting: BM25 narrows candidates, Jev re-filters by gate and fit score to the top 5, and Gemini makes the final pick. The comparison setting adds 2 more switches. Its gain was only +1.5pp, and it risked wrong answers by routing around low-fit skills, so I did not enable it.

Production rollout and the cost experiment

- **Letting BM25+Jev filter first is estimated to cut Gemini's cost to about 1/10.** After a small test, I reran the same 132 new questions. BM25 narrows 115 skills, Jev re-filters by gate and fit score, and only the top 5 reach Gemini. Gemini reads 5 skills instead of 115, so the estimated cost is about 1/10, while accuracy (119 of 132, 90.15%) was about the same as Gemini reading all 115. Jev returns probabilities, so it is faster and cheaper than Gemini (TypeSafe's published response time is 70-500ms; my measured median was 360ms). Narrowing cheaply and picking with a stronger model matches the direction of the published SkillRouter paper (arXiv 2603.22455). This is fully rolled out and is what Hermes's sub-agents use today. If Jev fails or times out, Gemini picks from the full list. Basis and scope: 115 skills, 132 unseen questions. The 1/10 cost is an estimate, not a measurement, from public unit prices and the number of skills Gemini reads.
- **Everything ran within the rate-limit quota.** I compared settings from saved answers with no extra calls, scaling from 20 to 90 to 132 questions. Rate-limit errors (429) stayed at 0 in every run.

What I turned on · what I decided

I confirmed the 3 changed code locations matched, backed up the config file, and added one line. I restarted the server and confirmed it was healthy. I rolled out the full pipeline (BM25 narrows, Jev re-filters, Gemini picks) only after 3 of 3 real questions came back correct. I set the bar before testing: clearly better on unseen questions, with no rise in wrong skill picks.

What I didn't turn on · limits

I tested a comparison setting with 2 more switches but did not ship it. The gain was about 1.5 points, and a confident model passed a skill with a fit score as low as 0.22, which is too risky.
Limits: the numbers come from a test environment, so they don't guarantee accuracy on all production traffic. Some test questions have more than one defensible answer. Nothing updates the answer key when a skill is added, so it had 10 labeling errors (5 in the existing set, 5 in the new questions). An automatic check is only an idea; I haven't built it.

LightRAG Knowledge Hub · Email Agent

StyleSeller · internal, private repository

Four agents find evidence in the same knowledge hub, and an email draft reaches staff's drafts folder only when it has evidence

Period	2026.03.31–present
Role	Design: workspace registry, search planning and evidence-judging steps, draft verification order / Build: email-agent server, checker, sent-mail learning cursor / Operate: draft-folder deployment, incident analysis and recovery
Repo	email-agent

Background

Vendor CS, Influencer CS, Spark, and sales-partner matching all share one LightRAG knowledge hub (13 workspaces). It's a RAG store that finds material for each question before answering. Every workspace carries a description, owner, channel, and sensitivity level, and using an unknown workspace fails immediately. For shared-mailbox inquiries, it puts a reply draft with evidence into the assigned person's Gmail drafts folder. Staff send it.

Structure and Scale

1. Tech

Python 3.12 · FastAPI · Pydantic v2, LightRAG 1.5.6 (Postgres/pgvector + Neo4j 5), RAG-Anything/MinerU, FlashRank (lightweight reranker), Gemini Flash family (via LiteLLM), Gmail, pytest

2. Scale (as of 2026-10-24)

- About 1,470 commits · about 47,000 lines of product code · about 33,600 lines of tests (10-24)
- 27 routers · about 1,750 test functions · 13 workspaces (10-24)
- 18 search evaluation cases (query · expected workspace · required evidence · forbidden claims)

3. How it works

① An LLM planner (temperature 0) looks at workspace descriptions and the data catalog and builds up to 12 queries. Plans that fail validation are dropped, and the rule-based router takes over.

② The search mode (mix, combining vector and graph search) is fixed in code at each call site. Agent search uses top_k (results to fetch) of 8 and caches the same query for 60 seconds.

③ Evidence is judged by rule. Real-time values like price, stock, and settlement are forbidden claims, and refund confirmations, contracts, and bulk sends get a human-approval marker. Only the missing evidence items get a second lookup.

④ Reranking (a step that re-orders results) groups results by topic. Within a group, official corrections and the latest documents go first, and results are interleaved one per group on output.

⑤ Up to 6 claims in a mail draft are checked against evidence by a checker and an LLM verifier. Claims without evidence are rewritten, and if unsupported claims still remain, the draft is dropped.

Incidents and Fixes

1. Rerank step lost (2026-06-19 to 07-23). On 06-18 the team switched to agentic RAG, a RAG in which the LLM plans the search. From the next day, the quality of CEO mail drafts dropped. Tracing the commit history showed that the rerank step was lost in that switch. It was restored on 07-23.

2. Search overload (07-10, 08-12). The Influencer CS bot held the whole shared connection pool, pushing even ordinary searches to 500s (07-10). A top_k 40 search triggered Neo4j overload warnings (analyzed 08-12). Split the connection pool per workspace and capped agent search at top_k 8.

3. Silent graph outage (2026-10-04). The container looked healthy even while Neo4j was down. Now /readyz sends Neo4j a RETURN 1 with a 2.5-second timeout and reports ok/fail, but this failure alone doesn't trigger a 503, so restarts don't loop. Search replies with 503 and Retry-After 30 after 2 attempts.

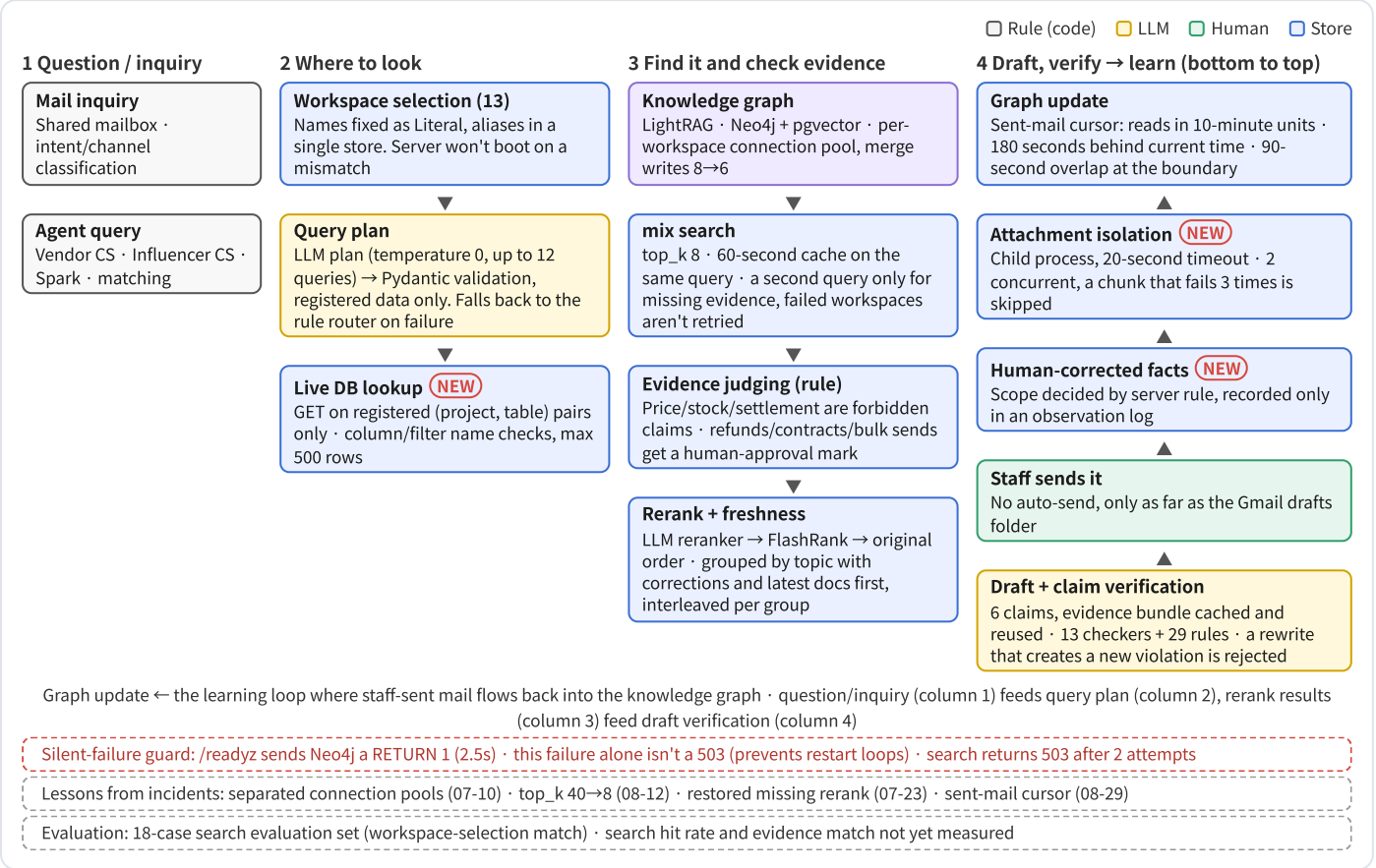
Status, Judgment, Limits

- Reply drafts go only as far as the assigned person's Gmail drafts folder. Sending is up to staff. Since 2026-08-18, only container images that pass pytest are deployed.
- Limit: the 18 search evaluation cases only check workspace selection. Search hit rate, evidence match, and the effect of restoring rerank are not measured.
- If I did it again: I'd add evidence-match scoring to the evaluation set first. That would have caught the missing rerank step before draft quality dropped.

What I Decided

- The LLM planner decides where to search; code decides the search mode. I didn't let the agent choose the mode too.
- The main source of truth is the mail staff actually sent. AI drafts are never fed back into the knowledge hub.
- Facts staff corrected aren't used right away — they go into an observation log first, to confirm the scope of the change.

Architecture



Sent-mail facts · KakaoTalk wiki · PDFs → knowledge hub (entities · relations) → LLM planner → search · evidence judging → draft · reply

Technical Highlights

1. The server won't boot if a workspace name is off. If agents called it by different names, writes could land in the wrong store, so the 13 canonical names are fixed as a Literal type and about 50 aliases are collapsed into a single store. On import, a missing entry or store mismatch stops startup with a RuntimeError.

2. An LLM search plan is only used after it passes validation. Plans are checked with Pydantic. Live DB lookups only allow registered (project, table) combinations, GET only, max 500 rows. If the call, parsing, or allowlist check fails, it falls back to the rule router and the reason is recorded as fallback:<exception name>.

3. If rerank fails, two more steps stand in. Documents are cut to 20 and the LLM reranker runs first. If that fails, FlashRank; if that also fails, the original order is kept, so a result always comes out. Concurrent search steps were lowered to 3, which cut concurrent Neo4j writes in the merge step from 8 to 6 per document.

4. 13 draft checkers each own one error pattern. Patterns like a reversed request direction, mentioning a prior mail that never existed, re-asking a settled condition, or stating a proposed price as if it were final — each checker has an actual mail reproduction case from 08-21 to 08-23. It started as regex, but false positives kept coming, so judging moved to an LLM checker.

5. A fix that introduces a new factual violation is rejected. Comparing only the sum of violation scores lets a 2-point factual violation pass in exchange for fixing a 10-point banned phrase. If a rewritten draft introduces a factual violation that wasn't there before, that rewrite is dropped even if it fixed something else. A draft that still has violations after rewriting is logged per violation type as "Draft rejected" and discarded.

6. The evidence bundle is cached and reused. Verifying 6 claims used to resend an evidence bundle of up to 150,000 characters each time. Now the bundle is cached for 5 minutes under a sha256 key and reused. If the cache fails, it falls back to the original method, so the judgment result doesn't change.

7. The sent-mail cursor accounts for Gmail's indexing delay. At deployment, 48 of 1,943 sent emails were left out of learning. Now it reads in 10-minute units and saves the cursor each unit. Since the search index lags, the cursor sits 180 seconds behind current time, and the boundary is read with a 90-second overlap. The cursor isn't advanced for a unit with more than 500 items.

8. The server decides the scope of a staff-corrected fact. Server rules judge whether a fact applies company-wide or to a single deal, so a single deal's fact isn't broadened to an entire partner. Results go only into an append-only observation log, run in shadow mode — never used in drafts or search.

Internal Work Tool · Pricing, Matching, Personalized KakaoTalk

StyleSeller · internal, private repository

Code calculates the amount; the LLM only returns candidates and the field to fix

Period	late March 2026 – present
Role	Design: price-calculation structure, matching-candidate merge rules, send-approval criteria. Build: pricing code, catalog extraction, matching, send automation, staff PC workers. Verify and operate: spreadsheet cross-checks, send/worker operations. (Sales-partner recommendation logic is owned by a colleague's CRM.)
Repo	work tool, server (staff PC workers)

Background

Pricing lived in Google Sheets formulas, and every quote was calculated by hand. While moving the formula into code, I added a test that checks it field by field against real products' sheet values. For catalog extraction and sales-partner matching, the LLM only produces candidates; staff register and review. A colleague's CRM decides the sales-partner-level recommendation for personalized KakaoTalk. I built the automation that receives those targets and sends.

Structure and Scale

1. Tech

React 19.2 · Tailwind 4 · Vitest 4, Postgres+PostgREST (REST server over tables), Gemini Flash family (via LiteLLM), knowledge-hub RAG search, kakaocli, staff PC workers

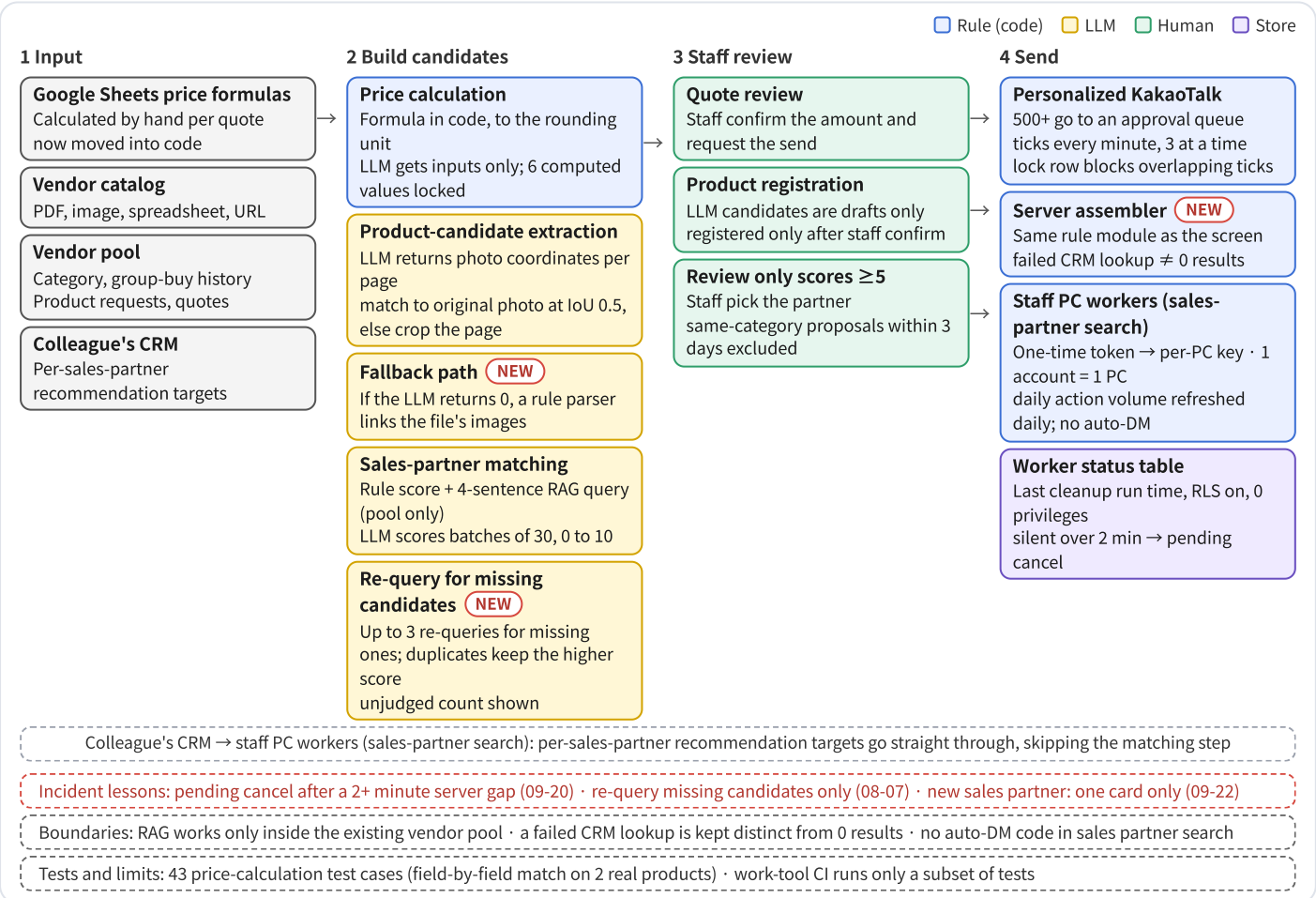
2. Scale (dated per item)

- Personalized KakaoTalk sent to 261 people in one day (2026-09-22)
- 43 price-calculation test cases, some cross-checked field by field against real products' sheet values
- Sales-partner search workers run on 9 staff PCs (2026-09-18)

3. How it works

① Pricing: sheet formulas moved into code, down to the rounding unit. On an edit request the LLM returns only the changed inputs as JSON, and code recalculates the amount.
② Catalog: the LLM pulls product candidates from PDFs, images, spreadsheets, and URLs. Only items staff confirm get registered.
③ Matching: rule-score and RAG candidates are merged, and the LLM scores batches of 30 from 0 to 10. Only scores of 5+ reach staff. Anything proposed in the same category within 3 days is excluded.
④ Personalized KakaoTalk: staff send, schedule, and stop from a screen. 500+ recipients go to an approval queue. Scheduled sends leave the server a few at a time every minute.
⑤ Sales partner search: staff PC workers run one PC per account, within a daily action cap. No auto-DM code was added.

Architecture



Incidents and Fixes

1. Missing matching candidates (2026-07-15). In 3 of 6 chunks, the LLM ended its response before judging every candidate. From 07-16, candidates are merged into a union first and then asked about in batches; from 08-07, up to 3 re-asks target only the missing candidates. The count still unjudged shows separately on screen.

2. Shared worker key (2026-09-10). Every staff PC worker shared a single key. Switched to issuing a per-PC key from a 30-minute, one-time install token. The server keeps only the key's sha256, compared with timingSafeEqual.

3. 6 rounds cancelled at once (2026-09-20). When the server came back after a two-day stop, the "worker unseen for 3 minutes" rule read the server gap as workers dropping out and cancelled 6 in-progress rounds at once. Now, if the cleanup interval exceeds 2 minutes, it's treated as a server gap and the reference time is rolled back.

Technical Highlights

1. I decided which fields the LLM can edit. If the LLM solved backward from a request like "set the sales partner's payout to 1,000," other amounts would shift too. So the 6 calculated values, like supply price and payout, are blocked from the edit list. For such requests, it only tells the user which input value to change — a request to change only the margin rate won't return a new sale price.

2. Moved the sheet formulas into code, down to the rounding unit. Partner supply price rounds up; the fee rate is fixed in code. Added a test that checks 2 real products from the sheet (e.g. a sample item at KRW 30,000) field by field. Also checks that total profit stays the same when the split ratio changes, and that the two shares always sum to total profit.

3. Looks for the original photo inside the PDF first. Sends each page to the LLM one at a time to get photo coordinates. If those coordinates overlap an original image embedded in the PDF by more than 0.5 IoU (the overlap ratio of two boxes), it uses the original. Backgrounds covering more than 80% of the page are excluded as candidates, and the page is only cropped when no matching original exists. A product spread across multiple pages has its empty price fields filled in from each other.

4. Excel still produces a result even if the LLM fails. If LLM parsing returns 0 items, a rule-based parser re-reads the same file instead of showing a blank screen. Images embedded in the file are attached to products in order. Icons under 750 bytes are dropped rather than treated as product photos.

5. RAG can only add candidates, never create new ones. Rule scores are ranked by category, group-buy history, product requests, and quotes, in that order, and 4-sentence RAG query results are added as a union. An ID RAG returns is only accepted if it's already inside the existing vendor pool. This logic used to live in n8n; moved it into code so it could be tested with vitest.

6. Automated sends and screen sends follow the same rule. The server assembler calls the same product-selection and template module as the screen, so the same person never gets a different product. A failed CRM lookup and 0 results are reported separately, and only 0 results get filled with popular products. The new sales-partner card is created only once — on 09-22, when a card was made per person, the renderer threw 980 instances of error 500.

7. Scheduled sends go out a little at a time each minute, without overlap. Targets with a personalized recommendation are assembled 3 per tick; targets without one, up to 30. If the unsent backlog reaches 5 or more, that tick is skipped. Once the send window has passed, it doesn't restart, so sends don't pile up overnight. A lock row per schedule prevents ticks from overlapping, and release requires the token set at claim time.

8. One account runs on only one PC. Checking and consuming the install token are combined into a single conditional UPDATE, so two PCs can't register with the same token at once. If the same Instagram account is claimed by two PCs, the one that logged in more recently wins. The server's work-assignment function follows the same rule. Daily action volume is redrawn within range every morning.

What I Decided

- I didn't leave amounts to the LLM. Code moved from the sheet does the calculation; the LLM only returns the input value to change.
- RAG search is only used to widen candidates. Vendors outside the existing pool can't get in.
- Sends of 500 or more don't go out immediately — they wait in an approval queue.

Staff PC Deployment · Operations

StyleSeller · internal, private repository

Deploys and operates KakaoTalk and Instagram work with a one-line install on 11 staff PCs (as registered on 2026-09-16)

Period	2026.02–present (Instagram DM expansion from 2026.02; KakaoTalk tool from 2026.05)
Role	Design: install, authentication, and update structure, plus a server-side reference date (the cutoff before which messages are dropped). Build: installer, workers, Windows collector, kakaocli fork changes. Operate: deployment, incident response, monitoring cron. I used an AI coding tool, set the scope, and checked the results myself
Repo	internal repo, kakaocli fork

Background

KakaoTalk and Instagram have no official API the server can call, so the work runs on staff Macs and Windows PCs. Install is one command, authentication uses a per-device key, and each tool updates on its own cycle. I run the server too. On 2026-08-20 a cloud DB limit stopped every work-tool API. I stood up the same API on a VM and recovered the same day.

Structure and Scale

1. Tech

macOS bash and Windows PowerShell installers, launchd and Task Scheduler, Node workers, Python collector (Windows), kakaocli fork (Swift), VM Postgres+PostgREST+nginx, SSE (server-push) hub, Docker deploy scripts, Slack monitoring cron

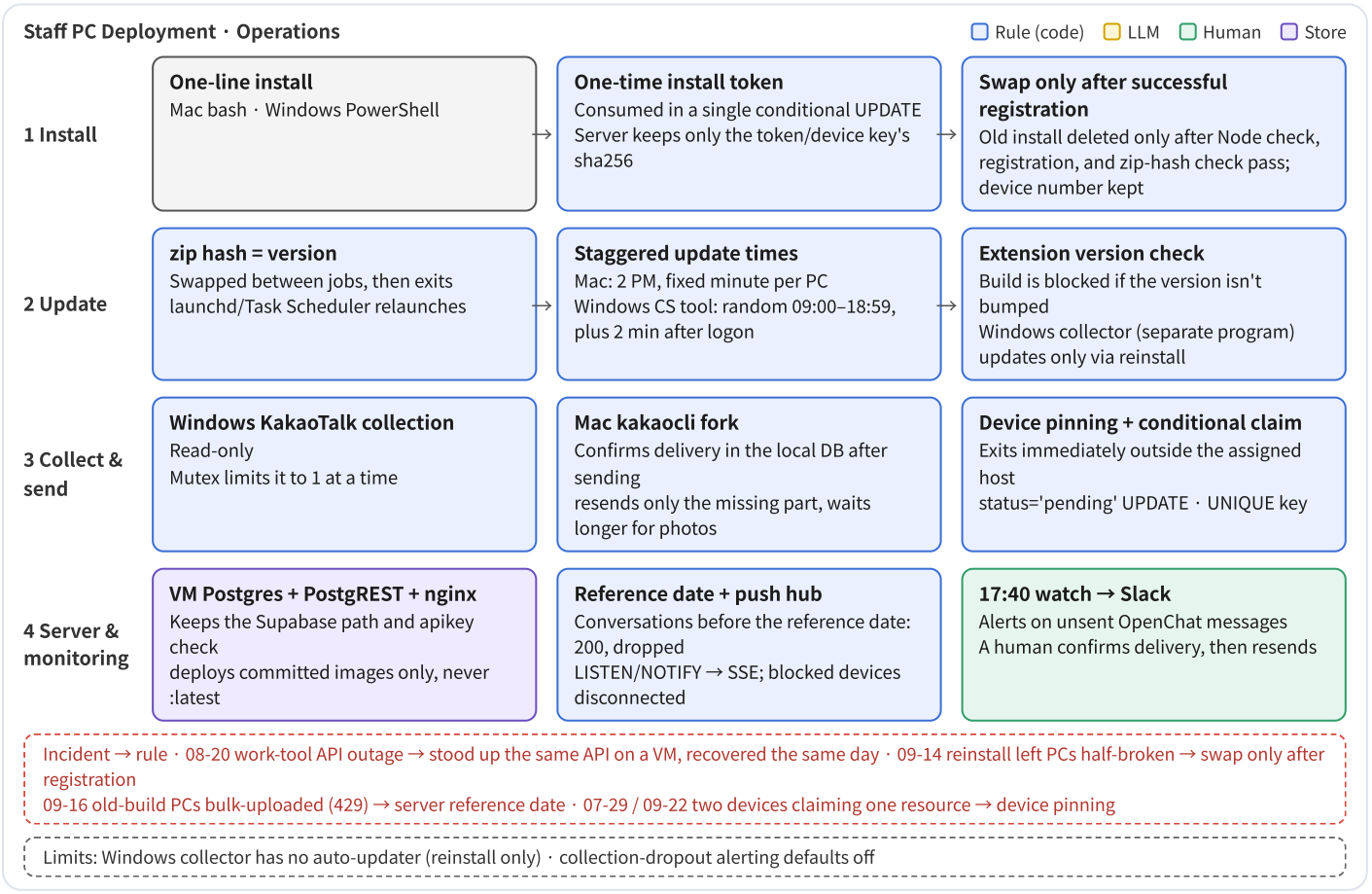
2. Scale (dated per item)

- 11 staff PCs (as of 2026-09-16 registration), sales-partner search worker PCs: 9 (09-18)
- Moved 4 DBs to self-hosting (2026-08-06 to 08-25)
- Recovered a full work-tool API outage the same day (2026-08-20)

3. How it works

① Staff install with a one-line command — bash on Mac, PowerShell on Windows.
② Registering with a one-time install token issues a per-device key. The server keeps only sha256 hashes of the token and key.
③ Updates are staggered per PC. The Mac tools and the Windows CS tool each auto-update on their own cycle; the Windows collector is a separate program and updates only by reinstall.
④ Only the assigned device picks up a queue job. Claiming is one conditional UPDATE, so two devices can never take the same job.
⑤ The server deploys only from committed images. A 17:40 watch alerts Slack to unsent OpenChat messages; a person confirms and resends.

Architecture



Incidents and Fixes

1. Full work-tool API outage (2026-08-20). When a cloud DB transfer limit stopped all work-tool APIs, a VM's nginx was set up to mimic the Supabase path and apikey header check exactly. The app only changed a connection setting and a few adapters, left about 100 createClient call sites in the app code untouched, and recovered the same day.

2. Reinstall broke working PCs (2026-09-14). The installer deleted the old install before registering the new one. Running an expired copy left some PCs with neither config nor package. Now the old folder is deleted only after Node check, registration, and zip-hash verification all pass. The device number is kept across reinstalls.

3. Old-build PCs uploaded their entire conversation history at once (2026-09-16). 3 Windows PCs sent their accumulated backlog all at once. The Sheets API got rate-limited (429), leaving the 2 PCs installed afterward with rows empty for over an hour. Fixing the client doesn't help until the PC gets the new build, so a reference date now lives on the server — messages before it are accepted with 200 and dropped.

Technical Highlights

1. The install token is consumed in a single UPDATE. Separating the lookup from the consume step would let two PCs register with one token at once. An UPDATE conditioned on used_at IS NULL and the expiry only passes when it returns a row. Failure reasons are split into staff-readable messages. If one device is compromised, only that row is discarded.

2. Uses a zip hash instead of a version number. The published zip's sha256 is the version. With no separate version ledger, the ledger and the file can never drift apart. The worker swaps only between jobs and simply exits; relaunching is left to launchd and Task Scheduler. If it relaunched itself and two copies ended up running, the Instagram profile would break.

3. Update times are staggered per PC. If every device updated in the same minute, there'd be no window to pull back a bad deploy. The Mac real-time sync picks a random minute at 2 PM and keeps it across reinstalls. The Windows CS tool fetches at a random time between 09:00 and 18:59, plus a retry 2 minutes after logon. The Windows collector is a separate program and updates only by reinstall.

4. Windows KakaoTalk uses a read-only collector. The collector only opens read access and never writes.

5. Send success is confirmed by the DB, not the exit code. On 2026-09-07, a send left out the text while the photo went through, and it was still marked sent. Now it reads the local KakaoTalk DB right after sending and immediately resends only the missing part. If a photo doesn't show up, it waits longer to allow for a slow upload — in a 70-person room, sending a duplicate photo is worse than missing one.

6. Device pinning and conditional claiming are two independent safeguards. On 2026-07-29, a dev Mac picked up the same queue as a remote-only Mac. Successes and failures alternated, and it took a day to find the cause. Now anything outside the assigned host exits immediately. A claim is a conditional UPDATE on status='pending', and the OpenChat queue is further guarded by UNIQUE(send_date, room_name).

7. Device blocks are delivered immediately via push. The push hub takes Postgres LISTEN/NOTIFY and wakes workers over SSE. If device_keys changes, it re-authenticates before waking. A blocked device gets a revoked signal and the connection is cut — the server side only needs to fill in one revoked_at field.

8. deploy.sh re-runs using the version of the commit being deployed. The 2026-09-28 deploy failure happened because the running script and the committed script differed. The image tag is fixed once as sha-<commit>, and if the pull fails, it stops instead of falling back to :latest. Compose is rendered before the build so a missing variable is caught before a 40-minute build, not after.

Status, Judgment, Limits

- In production. The 11 staff PCs and 9 sales-partner search worker PCs are counts as of the 2026-09-16/09-18 registrations; counts since then aren't recorded here.
- The most frequent incident was two devices claiming one resource (07-29, 09-22), which is why device pinning is the default.
- Limit: the Windows collector has no auto-updater, so it updates via reinstall. Collection-dropout alerting is off by default.
- If I did it again: I'd add a zip-hash updater to the Windows collector from day one. The 09-16 incident grew because old-build PCs never received the client fix.

What I Decided

- If there's no CI image, deployment stops instead of falling back to :latest, so a deployed image always matches a commit.
- Standing up the same API on a VM first, rather than fixing the app during the DB outage, meant the app-side change was just a connection setting and a few adapters.
- The reference-date filter never drops a message whose time it couldn't read. Seeing a little extra data is easier to notice and fix than data quietly disappearing.